Transactions

A database transaction groups a series of changes in such a way that either the database applies all of the changes or it applies none of the changes. The classic example of the need for transactions (and one used in Active Record's own documentation) is transferring money between two bank accounts. The basic logic is straightforward:

```
account1.deposit(100)
account2.withdraw(100)
```

But we have to be careful. What happens if the deposit succeeds but for some reason the withdrawal fails (perhaps the customer is overdrawn)? We'll have added $100 to the balance in `account1` without a corresponding deduction from `account2`. In effect, we'll have created $100 out of thin air.

Transactions to the rescue. A transaction is something like the Three Musketeers with their motto "All for one and one for all." Within the scope of a transaction, either every SQL statement succeeds or they all have no effect. Putting that another way, if any statement fails, the entire transaction has no effect on the database.

In Active Record we use the `transaction` method to execute a block in the context of a particular database transaction. At the end of the block, the transaction is committed, updating the database, *unless* an exception is raised within the block, in which case the database rolls back all of the changes. Because transactions exist in the context of a database connection, we have to invoke them with an Active Record class as a receiver.

Thus, we could write this:

```
Account.transaction do
  account1.deposit(100)
  account2.withdraw(100)
end
```